Tes Teknis Software Engineer SEVIMA — Project M2: Integrasi Pembayaran UKT via Webhook

Jalur: Middle/Senior · **Estimasi:** 12 sampai 16 jam

Tes teknis Software Engineer SEVIMA terdiri dari dua tahap: pengerjaan project (tahap 1) dan wawancara teknis dengan live coding berdasarkan project Anda (tahap 2). Selain fungsionalitas, kami menilai kematangan engineering: keputusan desain, kualitas kode, keamanan data antar-kampus, keandalan, pemahaman domain kampus, serta cara Anda bekerja bersama AI secara bertanggung jawab.

1. Latar Belakang

Banyak kampus menagih Uang Kuliah Tunggal (UKT) setiap semester melalui platform yang sama, dengan pembayaran lewat payment gateway pihak ketiga. Status pembayaran harus akurat karena menentukan apakah mahasiswa dapat mengisi KRS. Bangunlah aplikasi web yang mengelola tagihan UKT (halaman admin keuangan dan mahasiswa) dan menerima notifikasi pembayaran dari gateway.

2. Kebutuhan Wajib

Deliverable berupa aplikasi web fullstack: antarmuka pengguna (frontend), backend, dan database. Pengumpulan yang hanya berisi backend atau API tanpa antarmuka dianggap tidak lengkap.

1. Membuat **tagihan UKT** per mahasiswa per semester (nomor tagihan, nominal, tanggal kedaluwarsa) untuk setiap institusi.
2. Menyediakan **endpoint webhook** untuk menerima notifikasi dari payment gateway. 3. Memperbarui **status tagihan**: belum bayar, lunas, atau kedaluwarsa, sesuai notifikasi. 4. Menyimpan **riwayat transaksi** setiap tagihan.
5. Endpoint untuk melihat status tagihan dan riwayat transaksi seorang mahasiswa.
6. Dokumentasi endpoint webhook (format payload dan respons).

Halaman wajib (antarmuka pengguna):

- **Admin keuangan:** daftar tagihan institusinya dengan filter status, serta form membuat tagihan.
- **Mahasiswa:** status tagihan dan riwayat transaksi miliknya.

3. Aturan Bisnis

- **Sistem melayani banyak kampus dalam satu database.** Setiap data milik satu institusi (`institusi_id`), dan pengguna hanya boleh mengakses data institusinya sendiri.
- Setiap institusi memiliki **secret HMAC sendiri**. Notifikasi gateway membawa kode institusi dan signature HMAC-SHA256 pada header `X-Signature` yang harus diverifikasi dengan secret institusi tersebut. • Nomor tagihan unik di dalam satu institusi, tetapi bisa sama di institusi berbeda.
- Status tagihan harus mencerminkan kondisi pembayaran yang sebenarnya.
- Seluruh notifikasi yang diterima dicatat untuk keperluan audit.

4. Ketentuan Teknis

• **Stack yang diperbolehkan (standar SEVIMA):** Laravel (PHP), JavaScript/TypeScript (misalnya Node.js, Express, NestJS, React, Vue, Next.js, Nuxt), atau Go. Boleh berupa satu framework fullstack (misalnya Laravel dengan Blade atau Inertia; Next.js; Nuxt) maupun kombinasi backend dan frontend terpisah (misalnya Go + React, Laravel API + Vue). Untuk Laravel, **Livewire tidak diperbolehkan**. • **Database: PostgreSQL** (wajib).

- Project dapat dijalankan dengan langkah terdokumentasi (Docker Compose dianjurkan).

- **Sediakan** mock payment gateway beserta dokumentasinya ([tautan]): format payload, jenis event (`payment.success`, `payment.expired`), dan cara memicu webhook; seed data minimal tiga institusi. Secret HMAC mohon dikirim terpisah melalui email.
- **Di luar cakupan:** integrasi gateway sungguhan dan halaman pembayaran untuk mahasiswa (cukup status dan riwayat).
- Perilaku sistem pembayaran di dunia nyata tidak selalu ideal. Pertimbangkan hal tersebut dan tuliskan keputusan Anda di DECISIONS.md.

5. Bekerja Bersama AI

Anda **boleh dan dianjurkan** memakai AI coding assistant apa pun. Posisi yang kami harapkan: **Anda sebagai driver dan pengambil keputusan, AI sebagai navigator dan pengeksekusi.** Anda bertanggung jawab penuh atas setiap baris kode yang masuk ke repositori.

Yang tidak diperbolehkan: meminta orang lain mengerjakan, atau menyalin project orang lain.

6. Artefak Wajib

- **Kode project** yang dapat dijalankan.
- **README.md:** cara setup, menjalankan, dan mengakses aplikasi (termasuk akun demo untuk setiap role), serta asumsi.
- **CLAUDE.md** (atau file konteks AI lain): konteks project, aturan bisnis, konvensi, dan **batasan untuk AI, termasuk aturan akses data antar-institusi**.
- **AI_NOTES.md:** refleksi pairing dengan AI.
- **DECISIONS.md:** 3 sampai 5 keputusan teknis penting, alternatif, dan alasan.
- **Test otomatis** untuk logika bisnis inti, **termasuk isolasi data antar-institusi**.
- **Riwayat commit granular** dengan pesan bermakna. Secret tidak boleh tertulis di repositori.

Rules atau skill AI kustom opsional dan menjadi nilai tambah.

Template CLAUDE.md:

```
# Project: [nama project]

## Deskripsi singkat
## Stack dan cara menjalankan
## Arsitektur singkat
## Aturan bisnis penting (termasuk multi-institusi)
## Konvensi kode
## Hal yang tidak boleh dilakukan AI
```

Template AI_NOTES.md:

```
## Bagian yang dikerjakan bersama AI
## Satu contoh saran AI yang saya tolak/koreksi, dan alasannya
## Bagian yang saya kerjakan tanpa AI, dan alasannya
## Kalau mengulang dari awal, apa yang akan saya ubah
```

Template DECISIONS.md (per keputusan):

```
## [Judul keputusan]
Konteks :
Opsi :
Keputusan :
```

```
Konsekuensi :
```

7. Pengumpulan

- Repositori privat di GitHub atau GitLab dengan akses untuk nur.arifah@sevima.com, careers@sevima.com, alfian@sevima.com, dan mahendri@sevima.com
- Kirim tautan repositori ke email yang mengirimkan tes teknis sebelum tenggat dengan subjek “[Nama] TaskSE- SEVIMA”
- Untuk hal lain yang ambigu, tuliskan asumsi Anda.

8. Gambaran Tahap 2

Wawancara 75 sampai 90 menit secara daring dengan berbagi layar: demonstrasi aplikasi, penjelasan keputusan desain dan alur bisnis, analisis insiden dari log, pengembangan lanjutan bersama AI, code review, dan diskusi desain sistem. Siapkan project dalam kondisi dapat dijalankan dan terhubung ke mock gateway, serta AI coding assistant yang biasa Anda gunakan.